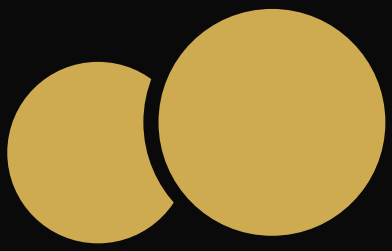

Izy Global Partners LLP

Library Management System
Legal & Knowledge Resources Centre

PROJECT MEMORY & HANDOFF RECORD
Definitive source of truth — Phase 1

Generated 2026-06-25

Table of Contents

1. Project Overview

The **Izy Global Partners LLP Library Management System** is an internal application for the firm's physical law library (the "Legal & Knowledge Resources Centre") in Abuja, Nigeria. It manages a single combined catalogue, member (borrower) records, loan tracking, and the firm's law-report serial runs.

Purpose & goals:

- Give the firm Librarian one tool to catalogue holdings, register borrowers, and track loans.
- Hold the catalogue to firm standards: five fixed groupings, accession numbers `IGP-LIB-00001`, editions kept separate, copies rolled up.
- Track law-report serials (NWLR Part-by-Part; LRECN and others by volume) with searchable indexes.
- Produce dashboards and Excel/print reports.
- Run locally with no external subscriptions; production target is PostgreSQL.

Hard rules (firm policy, enforced in code):

- Firm name renders **exactly** as `Izy Global Partners LLP` (never "IZY").
- The five groupings are the only catalogue groupings: Textbooks; Laws / Statutes; Legal Books / Essays / Commentaries; Law Reports; Reference Collections.
- Document references use the format `IGP / _____` (never insert "IOI").
- Single collection: `Izy Global Partners LLP`. (The earlier second collection "Alex. A. Izinyon & Co." was removed.)
- Firm authorship is reserved for the founder **Alex Izinyon**. Alex A. Izinyon SAN is NOT the founder — so no titles are auto-tagged; currently 0 books are firm-authored.

1.1 Two-phase plan

Phase 1 (BUILT): single operator — the Librarian (role `admin`) — does all cataloguing, member management, loans, import and reporting.

Phase 2 (ARCHITECTED, NOT BUILT): member logins (`members.linked_auth_uid`), member self-service (own history, holdings count, catalogue search, "apply for loan" via `loan_requests`), and a read-only Founder/Chairman (FC) dashboard. Roles `fc` and `member` and the `loan_requests` table exist but are inert. Textbook-selection authority is reserved to the FC by policy. See `ROADMAP.md`.

2. Architecture & Technical Design

```
Browser (React + Vite SPA, vanilla CSS)
   |  fetch JSON over /api ,  Authorization: Bearer <JWT>
   v
API server (Node + Express)  --  ALL business rules live here
   |  SQL
   v
Database:  SQLite now (better-sqlite3)  ->  PostgreSQL in production
```

- **Front end** holds no business logic about IDs/availability — it calls the API and renders. JWT kept in localStorage; a 401 clears it and returns to login.
- **API server** (`server/index.js`) owns auth, ID generation, duplicate roll-up, the loan lifecycle, merges, and the law-report/NWLR datasets. Every data route requires the `admin` role.

- **One source of truth: the database.** Derived values (copiesAvailable, loan Overdue status, NWLR held/available) are computed server-side so two devices always agree.
- **Dev runtime:** two processes — Vite (port 5173, HMR) + API (port 4000). Vite proxies `/api` to 4000.
- **Production runtime:** the API server also serves the built static app from `dist/`, same-origin on port 4000.

Note on the launcher: `Open IGP Library.command` runs `npm run dev` (port 5173) so the double-click experience matches manual dev exactly.

2.1 Technology stack & versions

Layer	Technology	Version
Frontend	React + ReactDOM	^18.3.1
Build/dev	Vite + @vitejs/plugin-react	^5.2.11 / ^4.3.1
Routing	react-router-dom	^6.23.1
Spreadsheet	SheetJS (xlsx)	^0.18.5
API server	Express	^4.19.2
DB (local)	better-sqlite3	^11.3.0
Auth	jsonwebtoken + bcryptjs	^9.0.2 / ^2.4.3
CORS / dev runner	cors / concurrently	^2.8.5 / ^8.2.2
Fonts	Cormorant Garamond + Jost (Google Fonts)	CDN, system fallback
Runtime	Node.js	24 (works on 18+)

3. Implemented Features & Status

Feature	Status	Notes
Auth (email/password, JWT)	Done	Admin only; 12h token; no public sign-up
Cataloguing add/edit/view	Done	All fields; validation; auto accession
Duplicate detection + copy roll-up	Done	title+author+edition+publisher match
Withdraw / mark missing (no hard delete)	Done	status change retains record
Delete record (with confirm)	Done	Delete-permanently OR Withdraw; blocked if on loan
Merge duplicate records	Done	POST /catalogue/merge — sums copies, deletes others
Authors list + merge	Done	derived from author strings; rename across catalogue
Author autocomplete in add form	Done	datalist of existing authors
Search + relevance ranking	Done	scope All/Title/Author; fuzzy + ranked; filters dropdown
Member management	Done	add/edit/active-inactive/notes; detail w/ loans
Loan issue / return / renew	Done	transactional; reference-only & 0-stock blocked
Overdue handling	Done	computed from dueDate vs today
Excel import (SheetJS)	Done	column mapping; dedupe; preserve accession
Dashboard	Done	totals, by grouping/collection, recent, overdue
Reports (7) + Excel export + print	Done	overview, current, overdue, history, acquisitions, members, NWLR
Law Reports tab (multi-series)	Done	NWLR (parts) + LRECN (volumes) + add more

Feature	Status	Notes
NWLR Parts held/missing + lookup + bands	Done	run to Part 2043; 810 held / 1233 missing
Law-report indexes (searchable)	Done	NWLR 10, LRECN 3 index entries seeded
Add report by part ranges; delete report	Done	"200-500, 502-771" bulk add; delete w/ confirm
Reusable DataTable + pagination	Done	10/page, numbered jump-to-page, sortable
Fixed sidebar + mobile hamburger drawer	Done	sidebar position:fixed; responsive drawer
Phase 2 (member/FC)	Stubbed	routes + roles exist; render "not available"

4. Database: Schema, Models, Relationships

Local schema: `server/schema.sqlite.sql` (applied on every server start, idempotent). Production target: `db/schema.postgres.sql` (mirrors it with native enums/identity). The server maps snake_case columns to camelCase JSON (`server/db.js`).

Table	Key	Purpose
users	email (unique)	Logins. Phase 1 = admin only. bcrypt password_hash.
settings	id=1	loanPeriodDays, renewalLengthDays, allowRenewals, firmName.
counters	name	Atomic sequences: accession / member / loan.
catalogue	accession_number (unique)	One row per bibliographic record.
members	member_id (unique)	Borrowers. linked_auth_uid = Phase 2 (null).
loans	loan_id (unique)	One row per loan; book title + member name denormalised.
law_report_series	abbreviation (unique)	NWLR (kind=parts) / LRECN (kind=volumes) / others.
law_report_volumes	id	Held/Missing volumes for volume-based series.
law_report_indexes	id	Searchable index entries per series.
nwlr_parts	part_no	One row per NWLR Part: Held / Missing.
nwlr_config	id=1	upper_bound (2043), upper_provisional(0), serial_accession.
loan_requests	id	PHASE 2 SCAFFOLD — unused in Phase 1.

4.1 Relationships & invariants

- `loans.accession_number` → `catalogue.accession_number`; `loans.member_id` → `members.member_id`. Loans denormalise book title + member name for fast lists.
- `law_report_volumes.series_id / law_report_indexes.series_id` → `law_report_series.id`; series links to its catalogue serial via `serial_accession`.
- copies_available is never < 0 and never > copies_total. A loan cannot be issued for Reference-only / Missing / Withdrawn items or when copies_available = 0.
- Issuing decrements availability (status → On loan at zero); returning restores it (→ Available). All transactional.
- Different editions = separate records; multiple copies of the same edition roll up into one record's copies_total.
- NWLR is ONE catalogue record; its ~2,000 Parts live in nwlr_parts so the title count is not inflated. Available Parts = Held Parts in [1, upper_bound].

4.2 Identifier strategy

Human IDs are formatted from integer counters incremented in the same transaction as the insert, so numbers never collide or reuse: `IGP-LIB-00001` (accession, 5-pad), `IGP-MEM-0001` (member, 4-pad), `IGP-LOAN-00001` (loan, 5-pad). On import, an existing accession is preserved and the counter bumped up via `ensureCounterAtLeast`.

4.3 Current data (seeded + Librarian edits)

Dataset	Count
Catalogue titles	142 (incl. NWLR + LRECN serials)
By grouping	Laws/Statutes 50, Textbooks 43, Legal Books/Essays 38, Law Reports 8, Reference 3
Members	1
Loans	0 (history cleared before handoff)
Law report series	2 (NWLR, LRECN)
LRECN volumes / indexes	32 / 3
NWLR Parts	2043 (Held 810, Missing 1233; run to Part 2043)
NWLR index entries	10 (Comprehensive Index vols)
Accession numbers	issued to IGP-LIB-00145; 142 in use. Retired (never reused): 00065, 00131, 00132.

Why 142 books but the highest number is 00145: the accession counter only ever counts up and never reuses a number. Deleting or merging a record retires its accession (leaving a gap) so a number always points to one specific item. The next new book will be IGP-LIB-00146.

Migrations are not formal files. Schema is applied idempotently on start; one-off data changes were run as node scripts (collection consolidation, firm-authorship clearing, loan-history clear, NWLR extension to 2043). The seeded SQLite DB `server/data/igp-library.db` is committed to git so clones get identical data.

5. API Endpoints

REST/JSON under `/api`. Auth: `POST /api/auth/login` returns a JWT (`{uid, email, role}`, 12h). All routes below require header `Authorization: Bearer <token>` and role `admin` (via `requireAdmin`). Secret: `IGP_JWT_SECRET`.

Method & path	Purpose
POST /api/auth/login	Authenticate → {token, user}
GET /api/auth/me	Resolve current user from token
GET / PUT /api/settings	Read / update settings
GET /api/catalogue	List all records
GET /api/catalogue/:id	One record
POST /api/catalogue	Create (accession auto unless preserved)
PUT /api/catalogue/:id	Update
POST /api/catalogue/:id/copies	Increment copy count (duplicate roll-up)
PATCH /api/catalogue/:id/status	Withdraw / missing / restore / reference
DELETE /api/catalogue/:id	Hard delete (blocked if copies on loan)
POST /api/catalogue/merge	Merge {keepId, mergeIds[]} — sum copies, delete others
GET /api/authors	Distinct author names + counts

Method & path	Purpose
POST /api/authors/merge	Replace {from[]} with {to} across catalogue
GET/POST /api/members ; GET/PUT /api/members/:id	Member CRUD
GET /api/loans	List loans (status computed)
POST /api/loans	Issue {bookId, memberId, dueDate?, notes?}
POST /api/loans/:id/return	Return
POST /api/loans/:id/renew	Renew (extend due date, ++renewedCount)
GET /api/nwlr/status	Held/missing counts, bands, flagged gaps
GET /api/nwlr/parts?status=	Parts list (Held/Missing) for export
GET /api/nwlr/part/:n ; POST .../hold	Lookup a Part ; flip Missing→Held
PUT /api/nwlr/upper-bound	Raise the run bound (new Parts default Held)
GET/POST /api/law-reports	List / create series (+ catalogue serial)
GET /api/law-reports/:id	Series detail (+ volumes)
GET/POST /api/law-reports/:id/indexes	Search / add index entries
POST /api/law-reports/:id/volumes (+ /bulk)	Add one volume / bulk by ranges
PATCH /api/law-reports/volumes/:vid	Toggle volume Held/Missing
DELETE /api/law-reports/:id	Delete series + volumes + indexes + serial record

Example — issue a loan:

```
POST /api/loans
Authorization: Bearer <token>
{ "bookId": 12, "memberId": 3, "dueDate": "2026-07-01", "notes": "" }
-> 201 { loanId: "IGP-LOAN-00001", status: "On loan", dueDate, ... }
```

6. Frontend Structure

- `src/main.jsx` → `App.jsx` (BrowserRouter). Routes guarded by `ProtectedRoute` (allow=[ROLES.ADMIN]) inside `Layout`.
- **State:** React local state + `context/AuthContext.jsx` (token-based auth: signIn/signOut, user, role, loading). No Redux. Data fetched per-page via `lib/*` API clients.
- **Routing:** / (Dashboard), /catalogue, /catalogue/new, /catalogue/:id, /catalogue/:id/edit, /search, /members(+new/:id/edit), /loans(+new), /law-reports(+/:id), /nwlr, /authors, /import, /reports, /settings, /login. Phase-2 stubs at /member/* and /fc/*.

6.1 Components & pages

File	Purpose
components/DataTable.jsx	Reusable sortable table; 10/page; numbered pagination
components/Pagination.jsx	Prev/Next + numbered jump-to-page (windowed)
components/Layout / Masthead / Sidebar	Shell: fixed sidebar, sticky masthead, mobile hamburger
components/Modal.jsx	Accessible modal (used by delete/add-series/duplicate)
components/IndexSearch.jsx	Search + add index entries for a law-report series

File	Purpose
components/StatusBadge / Spinner	Status pills; loading spinner
pages/Catalogue/*	List, BookForm (add/edit + duplicate modal + author picker), BookDetail (status, delete, merge)
pages/Search.jsx	Relevance search, scope, filters dropdown
pages/Members/*	List, MemberForm, MemberDetail (loans + history)
pages/Loans/*	LoanList (filters), IssueLoan (searchable clickable book picker)
pages/LawReports/*	LawReportsList (cards + add modal), LawReportSeries (volumes, ranges, delete)
pages/Nwlr/NwlrPage.jsx	Parts held/missing, bands, gaps, lookup, exports, indexes
pages/Authors/AuthorsPage.jsx	Author list + merge tool
pages/Reports/ReportsPage.jsx	7 reports, Excel export, print
pages/Import/ImportPage.jsx	SheetJS upload, column mapping, dedupe, bulk insert
pages/Settings/SettingsPage.jsx	Loan period, renewal, toggles
lib/*.js	API clients: api, catalogue, members, loans, nwlr, lawreports, settings, excel, format, constants
styles/theme.css / app.css	Brand tokens (black/gold, squared edges) + all component CSS

Design language: squared edges (no curves), editorial serif (Cormorant Garamond) headings + Jost UI font, logo gold `#ceaa51`. Masthead crest = `public/logo-crest.png` (cropped from the real two-disc logo). Login uses full `public/logo.png`.

7. Backend Structure

File	Purpose
server/index.js	Express app; all routes; auth middleware; business logic; serves dist/ in prod
server/db.js	better-sqlite3 connection; applies schema; ID generation; row→API mappers
server/schema.sqlite.sql	Local schema (idempotent CREATE TABLE IF NOT EXISTS)
server/createAdmin.js	Bootstrap/refresh the Librarian login (bcrypt)
server/seedHoldings.js	Import the 122 book holdings + NWLR serial/Parts (idempotent)
server/seedLawReports.js	Register NWLR series, create LRECN + volumes + indexes; textbooks
db/schema.postgres.sql	Production PostgreSQL DDL (enums, identity, FKs, indexes)

Business logic notes: role check is `requireRole('admin')` (structured to add fc/member in Phase 2). Loan issue/return are SQLite transactions. NWLR bands are generated dynamically in 500-Part chunks up to the upper bound. Range parsing ("200-500, 502-771") via `parseRanges()`.

8. Configuration, Environment & Deployment

Variable	Default	Purpose
IGP_API_PORT	4000	API / production server port
IGP_JWT_SECRET	igp-dev-secret-change-me	JWT signing secret — SET IN PRODUCTION
IGP_DB_FILE	server/data/igp-library.db	SQLite file location
PORT (Vite)	5173	Dev web server (vite.config.js)

- **Local install:** `npm install` (better-sqlite3 ships prebuilt binaries).
- **First admin:** `npm run create-admin -- <email> <password>`. No public sign-up.
- **Dev:** `npm run dev` → API :4000 + Vite :5173 (proxy /api). Double-click `Open IGP Library.command` does the same.
- **Production build:** `npm run build` → `dist/`; `node server/index.js` serves app + API same-origin on :4000.
- **Production DB:** CTO implements `db/schema.postgres.sql`, swaps the data layer in server/* to a pg client (same REST contract). Frontend unchanged.
- **Backups:** SQLite = copy the .db file; Postgres = pg_dump.

9. Third-Party Services & Libraries

- **SheetJS (xlsx)** — client-side Excel import (column mapping) and all report/list exports.
- **better-sqlite3** — synchronous embedded SQLite (dev/testing DB).
- **express, cors** — HTTP API. **jsonwebtoken, bcryptjs** — auth.
- **react-router-dom** — SPA routing. **concurrently** — runs API + Vite together.
- **Google Fonts** (Cormorant Garamond, Jost) loaded via CDN in `index.html`; degrades to Georgia/system if offline.
- **No external accounts, no Firebase, no analytics, no payment.** (Firebase was explicitly removed early on.)

10. Key Decisions, Assumptions & Tradeoffs

- **Dropped Firebase** for a local Node+SQLite backend (no subscriptions, runs offline) with a PostgreSQL schema for the CTO. The REST contract is the stable boundary.
- **NWLR as one serial + a Parts dataset** (not ~2,000 catalogue rows) keeps the title count honest; available = held Parts in range.
- **Authors stored as a semicolon-delimited string** on the record (no authors table). Author list/merge are derived operations — lighter, but author identity is by exact name.
- **Merge deletes the merged catalogue rows**; loan history keeps its denormalised title/accession, so history survives.
- **IDs are numbers in JSON** — the loan-issue bug was a string/number comparison; always compare with String() in selects.
- **The committed SQLite DB carries real data** so clones are identical; transient -wal/-shm are gitignored; checkpoint before committing.
- **Firm authorship is manual** and currently empty (founder is Alex Izinyon, not Alex A. Izinyon SAN).
- **Squared, black-and-gold luxury UI**; fixed sidebar; mobile hamburger; reusable DataTable with numbered pagination.

11. Outstanding Tasks, Known Issues & Roadmap

Known issues / technical debt:

- No automated tests yet (no unit/integration/e2e). Verification has been manual + one-shot builds.

- JS bundle > 500 kB (xlsx is large) — consider code-splitting / lazy import of the import/report pages.
- Author identity is by exact string; near-duplicate names need the merge tool (no fuzzy auto-merge).
- SQLite CHECK constraints on the existing committed DB still allow the old second collection value (harmless; data migrated). Fresh installs use the tightened schema.
- Loan issue/renew/return not yet click-tested in this session after the fix (compiles; logic corrected).
- No rate limiting / lockout on login; dev JWT secret must be overridden in production.

Roadmap (Phase 2 — see ROADMAP.md):

- Member logins via members.linked_auth_uid; enable ROLES.member route guards.
- Member self-service: own history, holdings count, catalogue search, "apply for loan" → loan_requests → Librarian approval.
- FC read-only dashboard (ROLES.fc); textbook-selection authority reserved to FC.
- Migrate to PostgreSQL (db/schema.postgres.sql) for multi-device firm-wide use.

12. Development Workflow, Testing & Release

- **Repo:** git, remote `origin` = github.com/ibogunjobi-dev/igp-library-software.git. The Librarian commits & pushes herself; the assistant does not push.
- **Commit flow for data edits:** stop the app (flush WAL), then `git add -A && git commit && git push` from the project root. The .db is tracked so app edits version with the code.
- **Testing:** currently manual via the running app + `npm run build` as a compile check. No test framework installed yet.
- **Release:** `npm run build` then serve via `node server/index.js`, or hand the Postgres schema + static build to the CTO.

13. Important Context & Decisions Log

Chronological context an engineer/AI must know (these were explicit user decisions during development):

- Firebase was removed in favour of local SQLite + a Postgres schema for the CTO.
- Real holdings imported from `IGP_Library_Catalogue.xlsx` (122 books) as already-registered; accession numbers preserved.
- NWLR missing-Parts list reconciles to exactly 1,233; run later extended to Part 2043 (1999–2043 added as Held).
- Logo: the two circles do NOT overlap into a blob — there is a real gap; the crest is cropped from the actual logo PNG.
- "Alex. A. Izinyon & Co." collection removed; founder is Alex Izinyon (not Alex A. Izinyon SAN) so no firm-authored titles.
- Project lives at `~/Development/igp-library` (moved from ~/igp-library).
- Loan history was cleared before handoff (0 loans, counter reset).
- LRECN added (32 volumes); LRECN indexes (3 ranges) + NWLR Comprehensive Index (10 vols) seeded.
- Every table was made a reusable component with 10/page numbered pagination; sidebar fixed; mobile hamburger added.

14. AI Continuation Brief

This section lets a fresh AI session resume immediately with no prior context.

Current project state

Phase 1 is functionally complete and runs locally. Frontend (React/Vite) + API (Express) + SQLite. Repo at `~/Development/igp-library`, remote set, the Librarian commits/pushes herself. Latest commit: "Loan issue fixed". The seeded SQLite DB is committed (clones get identical data).

What was completed

Auth; full cataloguing (add/edit/view/withdraw/delete/merge); duplicate detection + copy roll-up; author list/merge + autocomplete; relevance search with scope & filters; members; loans (issue/return/renew, overdue); Excel import; dashboard; 7 reports with Excel/print; Law Reports (NWLR Parts + LRECN volumes + indexes + add-by-range + delete); reusable paginated DataTable; fixed sidebar + mobile hamburger; black/gold squared luxury UI; correct cropped logo crest; loan history cleared.

In progress / not yet verified

- Loan-issue fix (string/number ID comparison) compiles but was not click-tested live in the last session.
- Uncommitted working-tree changes may exist (the user commits manually) — run `git status` first.

What to do next (suggested)

- Click-test issue/return/renew end to end; confirm availability + status transitions.
- Add a minimal test suite (Vitest for lib/* + a few API tests with supertest).
- Code-split heavy pages (import/reports) to shrink the bundle.
- When ready for multi-device: implement db/schema.postgres.sql and swap server/* data layer to pg.
- Begin Phase 2 only when asked (member logins, loan_requests flow, FC read-only).

Critical files

File	Why it matters
server/index.js	All API routes + business rules; start here for backend changes
server/db.js + schema.sqlite.sql	DB connection, ID generation, schema
db/schema.postgres.sql	Production DB target for the CTO
src/lib/*.js	API clients + pure helpers (constants, format, excel)
src/components/DataTable.jsx + Pagination.jsx	Every list uses these
src/pages/Loans/IssueLoan.jsx	Recently fixed; book picker + ID coercion
src/styles/theme.css + app.css	Brand tokens + all styling
server/data/igp-library.db	The live data (committed) — back up before edits
README.md / ROADMAP.md / IMPORT-NOTES.md / docs/TECHNICAL-OVERVIEW.md	Existing docs

Commands to run the project

```
cd ~/Development/igp-library
npm install
npm run create-admin -- librarian@izyglobalpartners.com "<password>"
npm run dev          # API :4000 + Vite :5173 (open http://localhost:5173)
#  or double-click 'Open IGP Library.command'
```

```
npm run build        # production static build into dist/
node server/index.js # serve app + API on :4000 (after build)
node server/seedHoldings.js "/path/IGP_Library_Catalogue.xlsx"  # (re)seed books
node server/seedLawReports.js                                    # seed law reports
```

Common pitfalls & implementation notes

- IDs are numbers in JSON but strings in form/select values — ALWAYS compare with String() (this caused the loan bug).
- Don't launch a dev server unless asked; the Librarian often runs the app herself. Prefer `npm run build` as a compile check.
- Do NOT commit or push unless asked — the Librarian handles git. Before she commits data edits, stop the app to flush the SQLite WAL into the .db.
- Firm name must be exactly "Izy Global Partners LLP"; only the five groupings; doc refs "IGP / ___".
- No firm-authored titles (founder = Alex Izinyon). Single collection only.
- NWLR is special (parts model + nwlr_config); other law reports use the volumes model. New parts-style reports use volume entries via the range bulk-add.
- better-sqlite3 is synchronous; loan issue/return run inside db.transaction(); keep Overdue computed, never stored.
- Schema is applied on every server start (idempotent). Adding a column to an existing committed DB needs a migration script, not just a schema edit.

End of project memory. This document plus the git repository fully reconstitute the project.